

MoiApp — Cloud Scalability & Load Testing Report

Architecture Review · DigitalOcean Deployment · Performance Testing Guide

1. Current Architecture Overview

MoiApp is built on a **Next.js 14 static frontend + PHP 8+ REST API + MySQL 8.0** stack. The frontend is statically exported (SSG) and served as flat HTML/CSS/JS files, making it extremely CDN-friendly. The PHP backend handles authentication, CRUD operations, file uploads, PDF generation, and bulk imports.

Layer	Technology	Role
Frontend	Next.js 14 (Static Export)	Serves event pages, dashboard, public client
API Backend	PHP 8+ (mysqli/PDO)	REST endpoints for events, moi, photos, auth
Database	MySQL 8.0	Stores users, events, moi_entries, photos, organizers
File Storage	Local disk (uploads/)	Cover photos, event photos (currently local)
Auth	JWT / Session tokens	User authentication & authorization

2. Current Flow (How Requests Are Handled)

Below is the request lifecycle for the two most critical user paths:

2.1 Public Guest — Viewing an Event & Adding Moi Entry

Step	Component	Action
1	Browser / CDN	User visits https://dsitesai.com/moiapp/e/wedding-slug
2	Nginx / Apache	Serves static index.html or event.html from Next.js out/
3	Next.js Static	Loads EventPublicClient.tsx (React hydrates)
4	API Call	GET /api/events.php?slug=wedding-slug → returns event details
5	API Call	GET /api/moi.php?event_id=123 → returns moi entries + breakdown
6	User Action	Guest fills form → POST /api/moi.php (no auth, slug-based)
7	PHP Backend	Validates input, inserts into moi_entries, returns success
8	Frontend	React updates UI with new entry (optimistic or refetch)

2.2 Admin — Creating Event & Managing Dashboard

Step	Component	Action
1	Browser	Admin logs in at /login → POST /api/auth.php?action=login
2	PHP Auth	Validates credentials, returns JWT/session token
3	Frontend	Stores token, redirects to /dashboard
4	Dashboard	Fetches ModuleEvents, ModuleAnalytics, ModuleMoi data
5	API Calls	Multiple parallel GET /api/events.php, /api/moi.php, /api/organizers.php
6	Admin Action	Creates event → POST /api/events.php (with auth header)
7	PHP Backend	Validates token, inserts event, generates slug, returns event
8	Frontend	Updates dashboard, navigates to event editor

3. Current Capacity Analysis

Based on code review of the PHP API endpoints and database schema:

Endpoint	Operation	Complexity	Bottleneck
GET /api/events.php?public=1	List all active events	Medium — subquery COUNT/SUM on moi_entries	Query slowdown
GET /api/events.php?slug=...	Single event detail	Low — indexed slug lookup	Minimal (fast with index)
GET /api/moi.php?event_id=...	List moi entries + breakdown	Medium — GROUP BY on event_id	Full table scan, no index
POST /api/moi.php	Add moi entry	Low — single INSERT	Minimal
POST /api/events.php?action=upload-photo	Upload event photo	Medium — file I/O + INSERT	DB I/O, file validation
GET /api/photos.php	List event photos	Low — simple SELECT	Minimal with index
POST /api/bulk-import.php	Bulk import moi entries	High — loops + multiple INSERTs	Transaction time, memory
GET /api/pdf.php	Generate PDF report	High — DOM parsing + PDF lib	PHP lib memory intensive

Key Finding: The public event listing uses subqueries (COUNT/SUM on moi_entries) for every row. With 10,000 events, this becomes slow. The moi_entries table lacks composite indexes on (event_id, created_at).

4. DigitalOcean Deployment Architecture

Two recommended approaches depending on your traffic expectations and budget:

4.1 Option A: Droplet (VPS) — Best for 0–10K Daily Users

A single managed VPS gives you full control and predictable costs. Recommended starting spec:

Component	Spec	Monthly Cost
Droplet	2 CPU, 2GB RAM, 50GB SSD (Ubuntu 22.04)	\$18

Managed MySQL	1 CPU, 1GB RAM, 10GB storage	\$15
Spaces (Files)	250GB storage + 1TB outbound	\$5
Total		\$38/month

Droplet Architecture Diagram:

Internet → Cloudflare (DNS + CDN) → Nginx on Droplet ■■■ Static files (Next.js out/) → served directly ■■■ /api/* → PHP-FPM → MySQL (Managed)
 ■■■ /uploads/* → Nginx (or proxy to Spaces)

4.2 Option B: App Platform — Best for 10K–100K Daily Users

Fully managed, auto-scaling, less ops overhead:

Component	Spec	Monthly Cost
Frontend (Static)	Auto-scaling, CDN included	\$0–\$5
Backend (Web Service)	1 CPU, 512MB → scales to 2 CPU, 1GB	\$12–\$24
Managed MySQL	1 CPU, 1GB RAM (Basic)	\$15
Managed Redis	256MB (optional, for caching)	\$10
Spaces (Files)	250GB + 1TB out	\$5
Total (min)		\$32/month
Total (with Redis)		\$42/month

App Platform Architecture:

Internet → App Platform Load Balancer ■■■ Frontend Component (static, CDN) ■■■ Backend Component (PHP, auto-scaling) ■■■ Managed MySQL (private network)

5. Load Testing Guide

Use **k6** (open-source, by Grafana) to test your DO deployment. Install it locally:

macOS: brew install k6
Linux: sudo apt install k6
Windows: choco install k6

5.1 Test Script: Public Event Browsing (Most Common Path)

Create **load-test.js**:

```
import http from 'k6/http'; import { check, sleep } from 'k6'; export const options = { stages: [ { duration: '30s', target: 50 }, // Ramp up to 50 users { duration: '2m', target: 50 }, // Hold 50 users { duration: '30s', target: 200 }, // Spike to 200 { duration: '2m', target: 200 }, // Hold spike { duration: '30s', target: 0 }, // Ramp down ], thresholds: { http_req_duration: ['p(95)<500', 'p(99)<1000'], // 95% under 500ms http_req_failed: ['rate<0.01'], // <1% errors }, }; export default function
```

```
() { // Test 1: Public event listing let res =
http.get(`${__ENV.BASE_URL}/api/events.php?public=1`); check(res, {
'events status 200': (r) => r.status === 200, 'events has data': (r) =>
r.json().length >= 0, }); sleep(1); // Test 2: Single event page res =
http.get(`${__ENV.BASE_URL}/api/events.php?slug=test-wedding`); check(res,
{ 'event status 200': (r) => r.status === 200, }); sleep(1); // Test 3: Moi
entries for event res =
http.get(`${__ENV.BASE_URL}/api/moi.php?event_id=1`); check(res, { 'moi
status 200': (r) => r.status === 200, }); sleep(2); }
```

5.2 Run the Test

```
# Test against your DO droplet k6 run -e BASE_URL=https://your-domain.com
load-test.js # Test against local MAMP first k6 run -e
BASE_URL=http://localhost:8888/MoiApp load-test.js # More aggressive test
(500 concurrent) k6 run -e BASE_URL=https://your-domain.com --vus 500
--duration 5m load-test.js
```

5.3 Interpreting Results

Good (passes thresholds): - p95 response time < 500ms - p99 response time < 1s - Error rate < 1% - No memory leaks (stable RAM usage) **Needs optimization:** - p95 > 1s → add indexes, enable OPcache, add Redis cache - Error rate > 1% → check PHP-FPM max_children, MySQL connections - High CPU → optimize queries, consider read replicas

6. Critical Optimizations Before Scaling

These changes have the highest impact on performance:

6.1 Database Indexes (CRITICAL)

Run these SQL commands on your DO Managed MySQL:

```
-- Composite index for moi entries (most queried) CREATE INDEX
idx_moi_event_created ON moi_entries(event_id, created_at DESC); -- Event
lookups CREATE UNIQUE INDEX idx_events_slug ON events(slug); CREATE INDEX
idx_events_user_created ON events(user_id, created_at DESC); -- Organizer
lookups CREATE UNIQUE INDEX idx_event_organizers ON
event_organizers(event_id, user_id); -- Photos CREATE INDEX
idx_photos_event ON photos(event_id, uploaded_at DESC);
```

6.2 PHP OPcache (HIGH IMPACT)

Edit `/etc/php/8.2/fpm/php.ini` on your Droplet:

```
opcache.enable=1 opcache.memory_consumption=128
opcache.interned_strings_buffer=8 opcache.max_accelerated_files=10000
opcache.validate_timestamps=0 ; set to 1 in dev, 0 in prod
```

6.3 Nginx Caching for Static Assets (MEDIUM)

Add to your Nginx config:

```
location ~* \.(jpg|jpeg|png|gif|ico|css|js|svg|woff|woff2)$ { expires 1y;
add_header Cache-Control "public, immutable"; } location ~* \.html$ {
expires 1h; add_header Cache-Control "public, must-revalidate"; }
```

6.4 Move File Uploads to DO Spaces (MEDIUM)

Replace local uploads/ with DO Spaces (S3-compatible). Update **config/s3.php**:

```
// Use DO Spaces endpoint $s3 = new Aws\S3\S3Client([ 'endpoint' =>
'https://nyc3.digitaloceanspaces.com', 'region' => 'nyc3', 'version' =>
'latest', 'credentials' => [ 'key' => env('DO_SPACES_KEY'), 'secret' =>
env('DO_SPACES_SECRET'), ], ]);
```

7. Monitoring on DigitalOcean

Set up these monitoring tools to track your app's health:

Tool	What to Monitor	Alert Threshold
DO Monitoring	CPU, Memory, Disk, Network	CPU > 80%, Memory > 85%
DO Managed MySQL	Connections, QPS, Slow queries	Connections > 80% max, Slow queries > 10/min
Nginx access log	Request rate, 4xx/5xx errors	5xx rate > 1%
k6 (scheduled)	API response times	p95 > 1s
Uptime Robot (free)	HTTP endpoint health	Down for > 2 min

8. Scaling Path

How to grow your DO infrastructure as traffic increases:

Traffic	Infrastructure	Monthly Cost	Action
0–5K users/day	2 CPU Droplet + Managed MySQL	\$33	Add indexes, OPcache
5K–20K users/day	4 CPU Droplet + Managed MySQL + Spaces	\$49	Add Redis, enable Nginx cache
20K–100K users/day	App Platform (auto-scale) + Managed MySQL + Redis	\$50–\$70	Add Cloudflare in front
100K+ users/day	Kubernetes + RDS + Redis + CDN	\$200+	Full microservices, read replicas

9. Step-by-Step DO Deployment Checklist

Follow these steps to deploy MoiApp on DigitalOcean:

1. Create DO account & add payment method
2. Create Managed MySQL database (Basic, 1 CPU, 1GB)
3. Create Spaces bucket for file uploads (region: nyc3)
4. Generate Spaces keys (Settings → API)
5. Create Droplet (Ubuntu 22.04, 2 CPU, 2GB, \$18/mo)
6. SSH into Droplet, install Nginx, PHP 8.2-FPM, MySQL client
7. Upload code: frontend/out/ → /var/www/moiapp/, api/ config/ uploads/ → same
8. Configure Nginx virtual host (see Section 4.1)

9. Create config/.env with DO DB credentials + Spaces keys
10. Import database: `mysql -u admin -p dbname < schema.sql`
11. Run SQL indexes (Section 6.1)
12. Configure PHP OPcache (Section 6.2)
13. Set up Cloudflare DNS pointing to Droplet IP
14. Enable DO Monitoring + set alerts
15. Run k6 load test against live URL
16. Verify: `https://your-domain.com, /api/auth.php?action=ping`

10. Cost Summary

Estimated monthly costs for different traffic tiers on DigitalOcean:

Scenario	Setup	Monthly Cost
MVP / Testing	Droplet + Managed MySQL	\$33
Small Launch	Droplet + Managed MySQL + Spaces + Redis	\$48
Growing (10K users/day)	App Platform + Managed MySQL + Redis + Spaces	\$52-\$72
Scale (50K+ users/day)	App Platform (auto-scale) + RDS + Redis + Cloudflare	\$150+

11. Quick Reference: DO Commands

Useful commands for managing your DO Droplet:

```
# SSH into droplet ssh root@your-droplet-ip # Check PHP-FPM status
systemctl status php8.2-fpm # Check Nginx status systemctl status nginx #
View PHP error log tail -f /var/log/php8.2-fpm.log # View Nginx error log
tail -f /var/log/nginx/error.log # Check MySQL connections mysql -u admin
-p -e "SHOW PROCESSLIST;" # Check slow queries mysql -u admin -p -e "SELECT
* FROM mysql.slow_log ORDER BY query_time DESC LIMIT 10;" # Restart
services after config changes systemctl restart php8.2-fpm systemctl
restart nginx # Check disk usage df -h # Check memory usage free -h
```